ALBUKHARY INTERNATIONAL UNIVERSITY

DU014 (K)

CRYPTOGRAPHY ESSENTIALS

PROJECT PROPOSAL

AES-Based Secure File Encryption and Decryption Tool

Student Name	_____Maralbyek Tilyek_____
Student ID	_____AIU24102280_____
Course / Course Code	Cryptography Essentials CCC2243
Project Type	Individual Project
Year / Semester	Year 2, Semester 3, 2025-2026

Table of Content

1. Background

Digital files routinely carry information that has real value to someone other than its owner: academic transcripts, financial statements, medical records, source code, or personal identification documents. When such files are stored on a laptop, a removable drive, or a shared network location without protection, anyone who gains physical or remote access to that storage medium can read, copy, or alter the contents freely. Encryption addresses this exposure directly by transforming readable plaintext into ciphertext that is computationally infeasible to reverse without the correct secret key, so that possession of the file alone no longer implies access to its contents.

The Advanced Encryption Standard (AES) is the symmetric-key block cipher formally adopted by the U.S. National Institute of Standards and Technology in FIPS PUB 197 (2001) after a multi-year public evaluation of candidate ciphers. It has since become the de facto global standard for protecting data at rest, underpinning technologies ranging from full-disk encryption (BitLocker, FileVault) to secure messaging and TLS. This project applies AES, specifically in its authenticated Galois/Counter Mode (AES-GCM) variant, to build a practical, correctly-implemented file encryption tool that a non-technical user can operate through a simple graphical interface, closing the gap between cryptographic theory and a usable end-user application.

2. Problem Statement

Despite the availability of strong, standardized encryption, most everyday users continue to rely on inadequate protection for sensitive files: hiding a file in an obscure folder, relying on operating-system-level password prompts that do not encrypt the underlying bytes, or using consumer archive tools whose default settings apply weak or outdated ciphers. None of these approaches provide genuine confidentiality against an attacker with direct access to the storage device, and few provide any mechanism to detect whether a file has been tampered with.

At the same time, many freely available "AES encryption" tools and code samples found online implement the algorithm incorrectly in ways that undermine its security guarantees entirely — most commonly by using Electronic Codebook (ECB) mode, which leaks structural patterns in the plaintext; by deriving the encryption key directly from a password without a proper key-derivation function, which makes offline brute-force attacks dramatically easier; or by omitting any integrity/authentication mechanism, which allows an attacker to modify ciphertext without the modification being detected on decryption. There is a clear need for a straightforward, correctly engineered application that gives ordinary users access to properly implemented AES encryption — one that protects confidentiality, resists brute-force password attacks, and detects tampering — without requiring them to understand the cryptography underneath it.

3. Aim

To design, implement, and evaluate a user-friendly desktop tool that encrypts and decrypts files using AES-256 in Galois/Counter Mode (AES-GCM) with a password-derived key, providing confidentiality, tamper detection, and password-based authentication in a single self-contained application.

4. Objectives

The specific objectives of this project are:

1. To research and document the theoretical foundations of AES, symmetric-key cryptography, password-based key derivation (PBKDF2), and authenticated encryption with associated data (AEAD), including why each design choice mitigates a specific class of attack.
2. To design the complete application workflow, including use case diagrams, a data flow diagram, and a flowchart covering both the encryption and decryption paths, before implementation begins.
3. To implement secure key derivation from a user-supplied password using PBKDF2-HMAC-SHA256 with a cryptographically random, per-file salt and an iteration count aligned with current NIST/OWASP recommendations.
4. To implement file encryption using AES-256-GCM, ensuring a unique random nonce is generated for every encryption operation and that the resulting authentication tag is verified before any decrypted output is written to disk.
5. To design and build a graphical user interface (Tkinter) that allows file selection via a dialog, password entry with masking and confirmation, a progress indicator for larger files, and clear, non-technical success and error messages.
6. To test the tool systematically across varied file types and sizes, correct and incorrect passwords, and deliberately corrupted ciphertext, verifying both functional correctness and the tool's ability to reject invalid decryption attempts.
7. To document the system design, implementation decisions, threat model, test results, known limitations, and possible future enhancements in a final report suitable for academic evaluation.

5. Project Scope

5.1 In Scope

The tool will provide local, single-user file encryption and decryption on a single computer. It will accept common file types including text documents, PDFs, images, and compressed archives, encrypting the file's raw bytes regardless of format. Every encrypted output file will be self-contained, storing the random salt, nonce, ciphertext, and GCM authentication tag required for later decryption, so that no separate key file needs to be managed by the user.

5.2 Out of Scope

- Cloud storage integration or network transmission of files.
- Multi-user account management, access control lists, or role-based permissions.

- Password recovery mechanisms (by design — a forgotten password must make the file unrecoverable, as this is a core security property, not a limitation).
- Enterprise-grade key management, hardware security module (HSM) integration, or centralized key escrow.
- Protection against attacks that require access to the running system's memory or an already-unlocked session (side-channel and cold-boot attacks).

6. Proposed Methodology

6.1 Encryption Workflow

1. The user selects a target file through a file-picker dialog and enters a password, which is confirmed by a second entry field to prevent typos.
2. The application generates a cryptographically random 16-byte salt using a secure random number generator (Python's secrets or os.urandom).
3. A 256-bit AES key is derived from the password and salt using PBKDF2-HMAC-SHA256 with a minimum of 480,000 iterations, consistent with current OWASP password-storage guidance adapted for key derivation.
4. The file's contents are read and encrypted with AES-256-GCM using a freshly generated 12-byte nonce, producing ciphertext plus a 16-byte authentication tag.
5. The salt, nonce, authentication tag, and ciphertext are concatenated in a defined binary header format and written to a new output file (e.g., document.pdf.enc), leaving the original file untouched unless the user opts to securely delete it.

6.2 Decryption Workflow

1. The user selects an encrypted file and enters the password used during encryption.
2. The application parses the salt, nonce, and authentication tag from the file header and re-derives the AES key from the supplied password using the stored salt.
3. The ciphertext is decrypted and the authentication tag is verified as part of the AES-GCM operation; if verification fails — because the password was wrong or the file was modified — the application aborts and displays an error without writing any output.
4. If verification succeeds, the original file is reconstructed byte-for-byte and saved under a name chosen by the user.

6.3 Threat Model and Security Considerations

The design assumes an attacker who may obtain a full copy of the encrypted file (e.g., a stolen laptop or leaked backup) but does not have access to the password and cannot observe the application's memory during use. Under this model, the combination of PBKDF2 (which slows down offline password-guessing) and AES-256-GCM (which provides both confidentiality and integrity) is intended to make the encrypted file computationally infeasible to decrypt without the correct password, and any tampering with the ciphertext detectable rather than silently accepted.

- Risk: weak or reused user passwords remain the weakest link regardless of cipher strength — mitigated by a password-strength indicator in the UI and a recommendation, not enforcement, of passphrase length.
- Risk: nonce reuse under the same key would catastrophically break AES-GCM's security guarantees — mitigated by deriving a fresh key (via a new salt) for every encryption operation, in addition to a fresh random nonce.
- Risk: incomplete or interrupted writes could corrupt output files — mitigated by writing to a temporary file and renaming only on successful completion.

7. Tools and Technologies

Component	Proposed Technology
Programming language	Python 3.10+
Graphical user interface	Tkinter (standard library)
Cryptography library	Python cryptography package (hazmat AEAD primitives)
Encryption method	AES-256-GCM (authenticated encryption)
Key derivation	PBKDF2-HMAC-SHA256, random salt, ≥480,000 iterations
Testing framework	pytest, for unit and round-trip integration tests
Version control	Git and GitHub
Documentation	Markdown, draw.io (for DFD/UML), Microsoft Word

8. Expected Outcome

The completed system will provide a working graphical interface for encrypting and decrypting arbitrary files. Encrypted output will reveal no discernible information about the original content, and any attempt to decrypt with an incorrect password or a modified ciphertext will fail cleanly with a clear error message rather than producing corrupted or misleading output. Testing will confirm, across a range of file types and sizes, that files successfully decrypted are byte-for-byte identical to their originals, and that encryption/decryption performance remains acceptable (target: under 5 seconds for a 100 MB file on typical consumer hardware). The final deliverable will include the complete source code, a documented test suite, and a report demonstrating that the implementation's security choices are grounded in established cryptographic standards.

9. Testing and Evaluation Criteria

Test Category	Criteria
Correctness	Encrypt-then-decrypt round trip reproduces the original file exactly, verified via hash comparison (SHA-256).
Security	Incorrect password is rejected; modified ciphertext or tag fails authentication and produces no output file.
Usability	A test user unfamiliar with the tool can encrypt and decrypt a file without external instructions.
Performance	Encryption/decryption time is measured and reported across file sizes from 1 KB to 500 MB.
Robustness	Application handles missing files, permission errors, and interrupted operations without crashing or corrupting data.

10. Proposed Work Schedule

Stage	Deliverable	Main Activities
1	Project Proposal (5%)	Define the problem, aim, objectives, scope, and tools.
2	Literature Review (10%)	Study AES, PBKDF2, salts, nonces, AEAD, and known implementation pitfalls.
3	System Design (10%)	Prepare use case diagram, DFD, flowchart, and UML class/sequence diagrams.
4	Implementation (30%)	Develop the encryption engine, file-header format, GUI, and error handling.
5	Testing and Evaluation (15%)	Run functional, security, usability, and performance tests; record results.
6	Final Report (20%)	Compile research, design, implementation, results, limitations, and conclusion.
7	Presentation and Demonstration (10%)	Prepare slides and demonstrate the working application live.

11. Conclusion

This project will demonstrate how modern authenticated symmetric cryptography can be applied correctly to protect real files against both disclosure and tampering. By combining AES-256-GCM with properly salted, iterated password-based key derivation, and by grounding every design decision in established standards (FIPS 197, NIST SP 800-38D, NIST SP 800-132, and current OWASP guidance), the proposed tool avoids the common implementation mistakes seen in many amateur encryption utilities while remaining simple enough for a non-technical user to operate. The result will be both a practical, portfolio-worthy application and a demonstration of applied understanding of the course's core cryptographic principles.

12. References (Preliminary)

National Institute of Standards and Technology. (2001). *Advanced Encryption Standard (AES)* (FIPS PUB 197). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.197

National Institute of Standards and Technology. (2007). *Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-38D

National Institute of Standards and Technology. (2010). *Recommendation for password-based key derivation* (NIST Special Publication 800-132). U.S. Department of Commerce. https://doi.org/10.6028/NIST.SP.800-132

OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). *Cryptography package documentation*. https://cryptography.io/